

Пояснительная записка

Проект: QuizArena - веб-приложение для проведения интерактивных опросов (квизов).

Курс: VK Education, Веб-приложение для проведения интерактивных опросов.
Контрольное занятие 1 - проектная задача.

1. Постановка задачи

Разработать MVP веб-приложения, позволяющего:

- организатору - создавать опрос-квиз, настраивать его и запускать комнату;
- участникам - подключаться к активной комнате по коду и проходить опрос синхронно с остальными;
- системе - определять победителей и сохранять историю прохождений для участников и организаторов.

Полный список функциональных требований - в задании курса.

2. Выбор средств разработки

Слой	Решение	Обоснование
UI	Next.js 14 (App Router), React 18, TypeScript	Единый фреймворк для SSR-страниц и API-маршрутов: уменьшает связки между фронтом и бэком, ускоряет MVP.
Стили	Tailwind CSS	Быстрая итеративная вёрстка без отдельных CSS-файлов; единый dark-стиль.
Realtime	Socket.IO 4	Зрелый протокол поверх WebSocket с автоматическим фолбэком на long-polling, простые room-broadcasts.
БД	SQLite + Prisma ORM	Один файл, ноль инфраструктуры - подходит для MVP. Prisma даёт типизацию схемы и миграции.
Аутентификация	JWT в httpOnly-куках + bcrypt	Не зависит от внешних сервисов, безопасно для CSRF (httpOnly + SameSite=Lax).
Загрузка файлов	Локальный диск (<code>public/uploads/</code>)	Без внешних зависимостей; для прод-режима меняется на S3 одной правкой.

3. Архитектура

Кастомный сервер в `server.ts` объединяет Next.js HTTP-обработчик и Socket.IO в одном процессе. Это даёт:

- общий порт для HTML/REST/WebSocket - никаких прокси при запуске;
- доступ к Prisma из обработчиков сокетов;
- простую модель деплоя (один Node-процесс).

Состояние идущих квизов хранится в двух местах:

1. **БД (SQLite)** - источник истины: пользователи, квизы, вопросы, комнаты, участники, итоговые ответы и баллы. Любой клиент может восстановить состояние, повторно подключившись.
2. **Память сервера** (`Map<roomCode, RoomRuntime>`) - таймеры, in-flight ответы текущего вопроса, набор подключённых сокетов. Это снимает нагрузку с БД на горячем пути и нужно только для синхронизации одного раунда.

В однопроцессной модели MVP этого достаточно. Для горизонтального масштабирования слой «runtime» переносится в Redis (готовый адаптер Socket.IO).

Поток одного раунда

```
host:next → [server] подбирает следующий вопрос,
            записывает Room.currentIndex/questionStart
            emits "quiz:question" всем в комнате
            ставит таймер на timeLimit + reveal

participant → "answer:submit" → server сохраняет в Map
            если все ответили – досрочный reveal

[server reveal] → начисляет баллы, пишет Answer/Participant.score,
            emits "quiz:result" с лидербордом

host:next → либо следующий вопрос, либо "quiz:final"
```

Подсчёт очков

`points * (0.5 + 0.5 * (1 - elapsed / timeLimit))` - половина баллов гарантирована за правильный ответ, вторая половина растворяется по мере прошедшего времени. Стимулирует скорость, не наказывая «поздних» драматически.

Множественный выбор засчитывается только при полном совпадении выбранных вариантов с эталонным множеством - простое и предсказуемое правило.

4. Этапы разработки

1. **Проектирование интерфейса.** Wireframes ключевых экранов (главная, вход, регистрация, кабинет, редактор, хост, участник) собраны в Excalidraw - см. ссылку в разделе 6.
2. **Модель данных.** Описана в `prisma/schema.prisma` и применена через `prisma db push`.
3. **Каркас фронта и API.** Next.js App Router: страницы `/login`, `/register`, `/dashboard`, `/dashboard/quiz/[id]`, `/host/[code]`, `/play/[code]`, `/me`. REST-маршруты в `src/app/api/*` для аутентификации, CRUD квизов и старта комнаты.
4. **Realtime-движок.** `src/lib/realtime/engine.ts` - машина состояний комнаты: `lobby` → `running` → `finished`, с таймером autoreveal и расчётом баллов в одной транзакции на вопрос.

5. **Личные кабинеты.** Чтение из БД, без отдельного API: `/dashboard` (свои квизы), `/dashboard/history` (история проведений), `/me` (история участия).
6. **Тестирование.** Полный сценарий «организатор регистрируется → создаёт квиз → запускает комнату → участник подключается и отвечает → видит финальный лидерборд» прогнан в браузере вручную. Подсчёт баллов проверен по записям в SQLite.

5. Соответствие требованиям

Требование	Реализация
Регистрация/авторизация	<code>/register</code> , <code>/login</code> , JWT cookie.
Создание и настройка квиза	<code>/dashboard/quiz/[id]</code> - название, описание, время и баллы на вопрос.
Вопросы разных типов	Поле <code>Question.type</code> $\in$ <code>single \ multiple</code> ; UI переключает radio/checkbox.
Картинки в вопросах	Загрузка через <code>/api/upload</code> , путь сохраняется в <code>Question.imageUrl</code> .
Подключение по коду	6-значный alfa-num код, страница <code>/play/[code]</code> .
Синхронный показ	Socket.IO broadcast <code>quiz:question</code> с серверным <code>startedAt</code> .
Подсчёт баллов	Скоринг с учётом скорости в <code>revealAndScore</code> .
Лидерборд	Событие <code>quiz:result</code> после каждого вопроса и <code>quiz:final</code> в конце.
Личные кабинеты	<code>/me</code> для участника, <code>/dashboard/history</code> для организатора.

6. Приложения

- **Репозиторий с кодом:** <https://github.com/floatxng/quiz-app>
- **Макеты (Excalidraw):** <https://excalidraw.com/#json=e3bUgAfZZTrEbJGMrwgGu,ysjQ0y3Adlgjp-L2Cpru4g>
- **Рабочий продукт:** запускается локально командой `npm install && npm run prisma:push && npm run dev` .